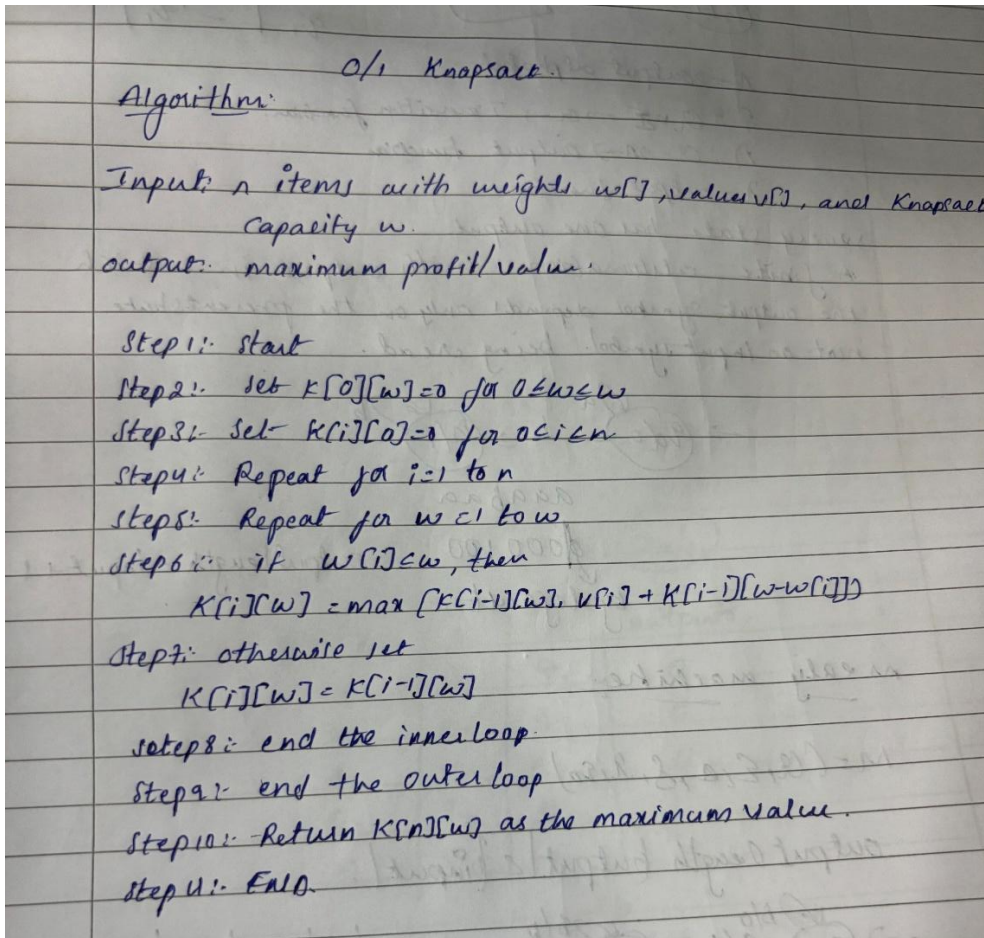


PRACTICAL-05

AIM: Implementation of 0/1 knapsack problem.

1.) KNAPSACK

ALGORITHM:



Algorithm: 0/1 Knapsack

Input: n items with weights $w[i]$, values $v[i]$, and Knapsack capacity w .

Output: maximum profit/value.

Step 1: Start

Step 2: Set $K[0][w] = 0$ for $0 \leq w \leq W$

Step 3: Set $K[i][0] = 0$ for $0 \leq i \leq n$

Step 4: Repeat for $i = 1$ to n

Step 5: Repeat for $w = 1$ to w

Step 6: If $w[i] \leq w$, then
$$K[i][w] = \max [K[i-1][w], v[i] + K[i-1][w-w[i]]]$$

Step 7: otherwise set
$$K[i][w] = K[i-1][w]$$

Step 8: end the inner loop.

Step 9: end the outer loop

Step 10: Return $K[n][w]$ as the maximum value.

Step 11: End.

PROGRAM:

```
#include<iostream>

using namespace std;

class Knapsack {
```

private:

```
int w[10];  
int v[10];  
int n;  
int W;  
int dp[10][100];
```

public:

```
void getInput() {  
    cout << "Enter number of items : ";  
    cin >> n;  
    cout << "Enter knapsack capacity : ";  
    cin >> W;  
    for(int i = 1; i <= n; i++) {  
        cout << "Item " << i << " weight and value : ";  
        cin >> w[i] >> v[i];  
    }  
}
```

```
void solve() {  
    for(int i = 0; i <= n; i++) dp[i][0] = 0;  
    for(int j = 0; j <= W; j++) dp[0][j] = 0;  
  
    for(int i = 1; i <= n; i++) {  
        for(int j = 1; j <= W; j++) {  
  
            if(w[i] <= j) {  
                // take or not take  
                dp[i][j] = max(  
                    v[i] + dp[i-1][j-w[i]],
```

```

        dp[i-1][j]
    );
} else {
    // cannot take item i
    dp[i][j] = dp[i-1][j];
}
}
}
}
}

```

```

void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "    ";
    for(int j = 0; j <= W; j++)
        cout << j << " ";
    cout << endl;

    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
        cout << endl;
    }
}

```

```

void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                << " (w=" << w[i]

```

```

        << ", v=" << v[i]
        << ") TAKEN " << endl;

        j = j - w[i];
    } else {
        cout << "Item " << i
            << " (w=" << w[i]
            << ", v=" << v[i]
            << ") NOT taken " << endl;
    }
}

// — Show result —
void showResult() {
    cout << "\nMaximum Value = "
        << dp[n][W] << endl;
}

};

int main() {

    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();

    return 0;
}

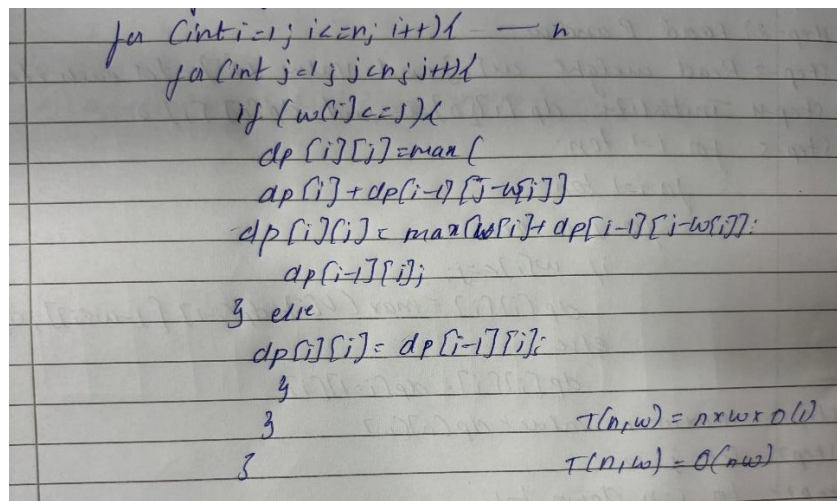
```

OUTPUT:

```
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAAG> cd "c:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAAG" ; if ($?) { g++ knapsackproblem.cpp -o knapsackproblem } ;  
if ($?) { .\knapsackproblem }  
Enter number of items : 4  
Enter knapsack capacity : 8  
Item 1 weight and value : 2  
1  
Item 2 weight and value : 3  
2  
Item 3 weight and value : 4  
5  
Item 4 weight and value : 5  
6  
  
DP Table:  
0 1 2 3 4 5 6 7 8  
i=0 0 0 0 0 0 0 0 0  
i=1 0 0 1 1 1 1 1 1 1  
i=2 0 0 1 2 2 3 3 3 3  
i=3 0 0 1 2 5 5 6 7 7  
i=4 0 0 1 2 5 6 6 7 8  
  
Maximum Value = 8  
  
Selected Items:  
Item 4 (w=5, v=6) TAKEN  
Item 3 (w=4, v=5) NOT taken  
Item 2 (w=3, v=2) TAKEN  
Item 1 (w=2, v=1) NOT taken  
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAAG>
```

TIME COMPLEXITY:

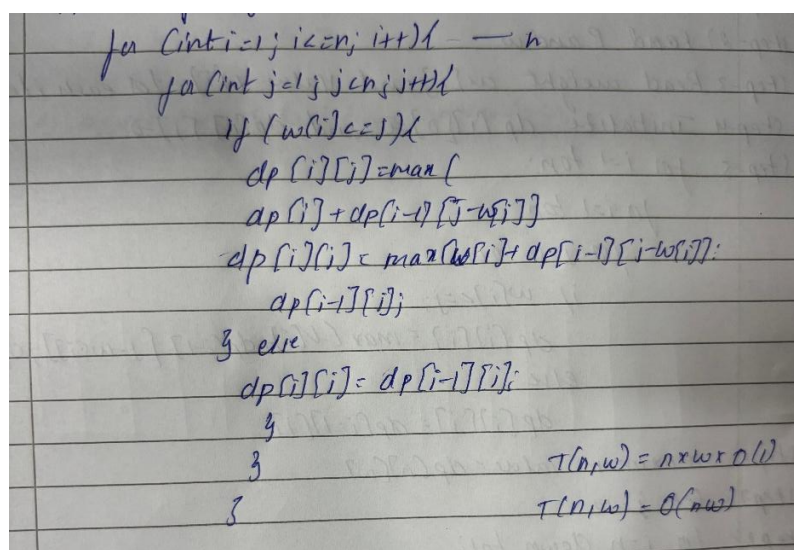
Best case:



```
fn(int i=1; i<=n; i++) {  
    fn(int j=1; j<=j; j++) {  
        if (w[i] <= j) {  
            dp[i][j] = max(  
                dp[i] + dp[i-1][j-w[i]]  
                dp[i-1][j]  
            );  
        } else {  
            dp[i][j] = dp[i-1][j];  
        }  
    }  
}
```

$T(n, w) = n \times w \times O(1)$
 $T(n, w) = O(nw)$

Average case :



```
fn(int i=1; i<=n; i++) {  
    fn(int j=1; j<=j; j++) {  
        if (w[i] <= j) {  
            dp[i][j] = max(  
                dp[i] + dp[i-1][j-w[i]]  
                dp[i-1][j]  
            );  
        } else {  
            dp[i][j] = dp[i-1][j];  
        }  
    }  
}
```

$T(n, w) = n \times w \times O(1)$
 $T(n, w) = O(nw)$

Worst case:

```
for (int i=1; i<=n; i++) {
    for (int j=1; j<=W; j++) {
        if (w[i]<=j) {
            dp[i][j] = max(
                dp[i-1][j],
                dp[i-1][j-w[i]] + v[i]
            );
        } else {
            dp[i][j] = dp[i-1][j];
        }
    }
}
```

$T(n, w) = n \times w \times O(1)$
 $T(n, w) = O(nw)$

Conclusion – Knapsack

The Knapsack Problem is an important optimization problem in computer science used to select the most valuable items while keeping the total weight within a given capacity. Using Dynamic Programming, the problem can be solved efficiently by considering the optimal combination of items. It helps in understanding optimization, decision-making, and resource allocation, and has practical applications in areas such as investment planning, cargo loading, budgeting, and project selection.